

Arquitectura de Microservicios con Contenedores

Docentes

- Ing. Federico Castañeda
- Ing. Carlos Maceira
- Ing. Franco Rupnik

Linux y contenedores

Namespaces · cgroups · OverlayFS · runtime stack

Objetivos de la clase

Al finalizar esta clase vas a poder:

- **Explicar** por qué el kernel de Linux no tiene un objeto "container"
- **Identificar** qué aísla cada namespace y qué limita cada cgroup
- **Describir** cómo OverlayFS materializa los layers de una imagen
- **Comparar** VMs vs containers a nivel kernel
- **Recorrer** el runtime stack completo (docker → run → containerd → runc → kernel)
- **Reconocer** las Linux capabilities y cuáles mantiene Docker por defecto
- **Evaluar** el trade-off del kernel compartido (gVisor, Kata)

El problema

El 76% de las organizaciones corre containers en producción, pero muy pocos equipos pueden explicar qué es un container para el kernel.

Industry survey, 2025

Warm-up 1 / 6

Linux no tiene un objeto "container" nativo en el kernel.

¿Qué primitivas del kernel se combinan para crear lo que llamamos un container, y qué rol cumple cada una?

Warm-up 2 / 6

"Container = proceso" es algo que se dice seguido en Linux.

¿En qué sentido esa afirmación es correcta, y de qué maneras importantes puede ser engañosa al diseñar sistemas productivos?

Warm-up 3 / 6

Los namespaces dan isolation, pero no resource fairness.

¿Por qué los namespaces por sí solos son insuficientes para workloads multi-tenant, y cómo cambian el comportamiento los control groups?

Warm-up 4 / 6

Una container image está hecha de layers inmutables.

Conceptualmente, ¿cómo obtiene un container en ejecución un filesystem de escritura a partir de esos layers read-only, y por qué es útil?

Warm-up 5 / 6

Los containers comparten el kernel del host.

¿Cuáles son las implicancias arquitectónicas y de seguridad de este diseño, comparado con virtual machines que corren kernels separados?

Warm-up 6 / 6

Expliquen cómo se materializa una OCI image en el filesystem de un host Linux.

Expliquen, en detalle, cómo una secuencia de image layers queda disponible para un container.

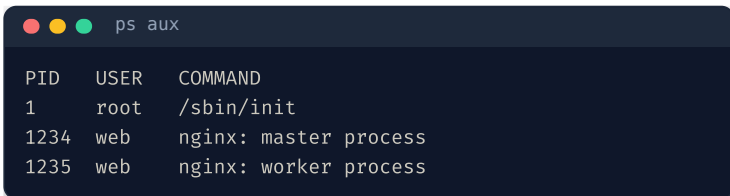
Cómo Linux construye un container

Namespaces · cgroups · OverlayFS

Linux: todo es un proceso

Un container **no es una VM chica**. Es un proceso del host con una vista restringida del mundo.

Sin kernel nuevo, sin emulación – solamente aislamiento (*isolation*).



```
ps aux
```

PID	USER	COMMAND
1	root	/sbin/init
1234	web	nginx: master process
1235	web	nginx: worker process

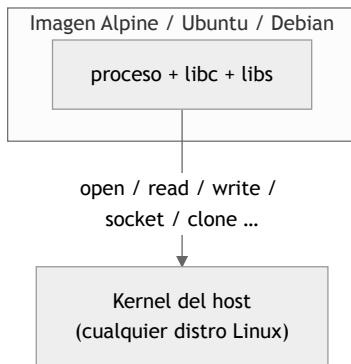
El container es el proceso 1234.

Userspace de la imagen, kernel del host

La imagen trae **userspace** (binarios, libc, apt/apk)
– **no** trae un kernel.

Cuando el proceso necesita I/O, red o memoria, hace un **system call**. Ese syscall lo atiende el **kernel del host**.

Por eso un container Alpine corre en Ubuntu, RHEL o Debian: todas hablan el mismo contrato Linux (ABI de syscalls).



La distro del host y la de la imagen **no tienen que coincidir**.

Fuente: kernel.org – ABI/stable/syscalls

Namespaces: achicar el mundo

Namespace	Aísla
pid	Árbol de procesos (el container se ve como PID 1)
net	Network stack, interfaces, rutas
mnt	Mount points del filesystem
uts	Hostname y domain name
ipc	Shared memory, message queues
user	Mapeo de UID/GID (root adentro $\neq$ root afuera)

Namespace: process isolation

host vs container

El host ve:

PID 1234: nginx (proceso del container)

El container ve:

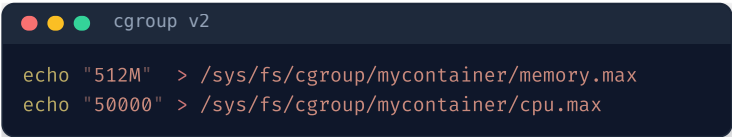
PID 1: nginx # su propio PID namespace

cgroups: limitar el apetito

Los **namespaces** limitan **qué** se ve.

Los **cgroups** limitan **cuánto** se usa.

Cada flag `--memory` y `--cpus` de `docker run` escribe en `/sys/fs/cgroup`.

A terminal window with a dark background and light text. The title bar shows three colored circles (red, yellow, green) and the text 'cgroup v2'. The terminal contains two lines of text: 'echo "512M" > /sys/fs/cgroup/mycontainer/memory.max' and 'echo "50000" > /sys/fs/cgroup/mycontainer/cpu.max'.

```
cgroup v2  
echo "512M" > /sys/fs/cgroup/mycontainer/memory.max  
echo "50000" > /sys/fs/cgroup/mycontainer/cpu.max
```

cgroup: resource enforcement

```
docker run --memory=512m --cpus=0.5 myapp
```

RAM CPU

En un ambiente de CI con varios jobs concurrentes, los cgroups evitan que una suite pesada deje sin recursos a las demás. Es también la razón por la que importan los resource requests/limits en Kubernetes.

Isolation: dos primitivas

Namespaces – qué ves

- PID, Network, Mount, IPC, UTS, User

El container no puede hacerle `kill` a procesos del host: ni siquiera puede verlos.

cgroups – qué *consumís*

- CPU, Memory, PIDs, I/O

Previene noisy neighbor en runners compartidos y fork bombs vía `pids.max`.

OverlayFS: union mount en el host

OverlayFS **no copia** los layers: apila directorios del host y expone una vista unificada en un **mount point nuevo**.

Directorio	Rol en el host
lowerdir	Layers de la imagen – read-only , compartidos entre containers
upperdir	Diff writable del container (escrituras y whiteouts)
workdir	Scratch interno del kernel (mismo filesystem que upperdir)
merged	Mount point donde el kernel monta el overlay y se ve el resultado

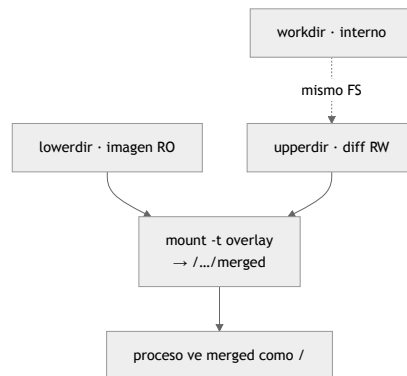
El proceso del container ve solo merged (su /). En el host siguen existiendo lower/upper/work como dirs normales.

Cómo se monta en el host

```
mkdir -p /tmp/ov/{lower,upper,work,merged}
echo 'app' > /tmp/ov/lower/index.js
```

```
mount -t overlay overlay \
-o lowerdir=/tmp/ov/lower,\
  upperdir=/tmp/ov/upper,\
  workdir=/tmp/ov/work \
  /tmp/ov/merged
```

merged = mount point del overlay (ahí se ve el merge).



Escrituras → upper · borrados → whiteout · lower intacto

OverlayFS: layers apilados

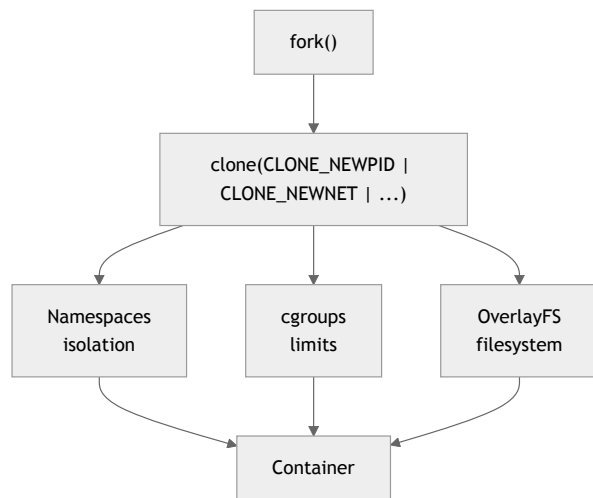
OVERLAYFS • UNION MOUNT

Escribir app.log Borrar index.js Reset

LOWER	UPPER	MERGED
image · read-only	container · writable	lo que ve el proceso
	• vacío	

Borrar un archivo de la imagen agrega un **whiteout**. Los layers inferiores no se modifican.

Las tres primitivas juntas



VMs vs containers

Por qué el mismo hardware rinde distinto

VMs vs containers

	VM	Container
Kernel	Guest kernel	Host compartido
Boot	20-60 s	50-500 ms
Overhead de RAM	~1 GB	~15-20 MB

Una VM lleva **su propio sistema operativo** completo: kernel, init, drivers virtuales.

Un container lleva **solo tu proceso** y sus dependencias de userspace.

Hardware vs OS virtualization

Las VMs emulan **hardware**. Los containers aíslan **procesos**.



Densidad: el mismo hardware

Servidor con 128 GB de RAM

VMs

60-80

instancias (footprint mínimo)

Containers

400-500

instancias · ~8× densidad

Comparación de CPU overhead

Workload	VM (KVM)	Container
Nginx	15.8%	1.7%
PostgreSQL	13.9%	1.4%
Storage (100 instancias)	~2 TB	~1.5 GB

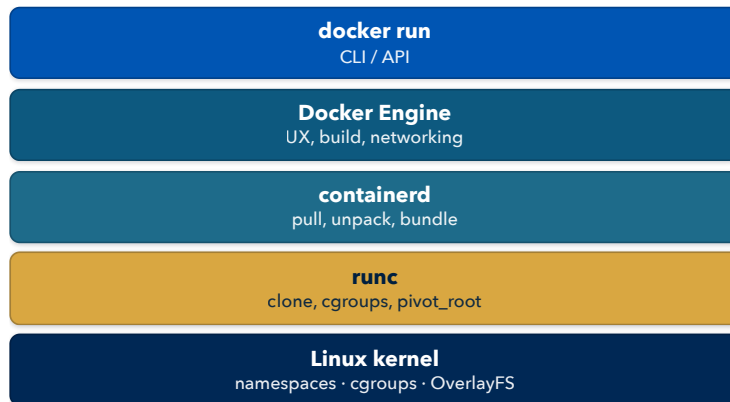
El runtime stack

`docker run` → Engine → containerd → runc → kernel

OCI: el contrato estándar

- **OCI Runtime Spec** – cómo correr un container
- **OCI Image Spec** – cómo empaquetarlo

→ Build once, run en Docker, Kubernetes, Podman, etc.



Fuente: <https://opencontainers.org>

El trabajo de containerd

1. Hace pull de los image layers desde el registry
2. Los desempaqueta → root filesystem
3. Genera el `config.json` (namespaces, capabilities, limits)
4. Invoca a runc con el bundle

runc: el que le habla al kernel

```
runc exec:  
  1. Parsea config.json  
  2. Crea namespaces (syscall clone)  
  3. Configura cgroups vía systemd  
  4. Monta rootfs + proc/sysfs  
  5. pivot_root ← chroot seguro  
  6. Aplica seccomp + AppArmor/SELinux  
  7. Dropea capabilities  
  8. exec() → tu proceso
```

Security hardening en runc

seccomp

Bloquea syscalls peligrosas

AppArmor / SELinux

Política MAC incluso para root

Capability drops

Principio de menor privilegio

Capabilities: root de grano fino

Históricamente, **root = todos los privilegios**. Linux los parte en bits independientes: las *capabilities*.

Idea	Detalle
Qué son	Privilegios del kernel (CAP_*) que un proceso puede tener o no
Por qué importan	Un proceso "root" en el container no es root completo del host
Cómo se controlan	runc dropea; Docker expone --cap-add / --cap-drop
Default Docker	Conserva 14 ; dropea el resto (--privileged da todas)

Ejemplo canónico: CAP_NET_BIND_SERVICE permite bindear el puerto 80 **sin** ser root absoluto.

Capabilities que Docker mantiene (14)

Capability	Qué permite
CHOWN	Cambiar owner (UID/GID) de archivos
DAC_OVERRIDE	Ignorar permisos de lectura/escritura/ejecución
FOWNER	Bypass checks que exigen ser dueño del archivo
FSETID	Conservar bits setuid/setgid al modificar un archivo
KILL	Enviar señales a procesos ajenos
MKNOD	Crear device nodes con mknod (2)
NET_BIND_SERVICE	Bindear puertos privilegiados (menores a 1024)
NET_RAW	Sockets RAW/PACKET (p. ej. ping)
SETGID / SETUID	Cambiar GIDs / UIDs del proceso
SETFCAP	Asignar capabilities a archivos
SETPCAP	Modificar el set de capabilities del proceso
SYS_CHROOT	Usar chroot (2)
AUDIT_WRITE	Escribir al log de auditoría del kernel

Capabilities que Docker dropea (1/2)

Capability	Qué permite
SYS_ADMIN	Admin del sistema (mount, namespaces, ...) – la más amplia
SYS_MODULE	Cargar / descargar módulos del kernel
SYS_RAWIO	I/O de puertos (iopl / ioperm)
SYS_PTRACE	Trazar procesos ajenos con ptrace(2)
SYS_BOOT	reboot / kexec_load
SYS_TIME	Cambiar el reloj del sistema
SYS_NICE	Subir prioridad / nice de procesos
SYS_RESOURCE	Override de resource limits
SYS_PACCT	Process accounting (acct(2))
SYS_TTY_CONFIG	Configurar terminales virtuales
SYSLOG	Operaciones privilegiadas de syslog(2)
NET_ADMIN	Configurar interfaces, rutas, firewall

Capabilities que Docker dropea (2/2)

Capability	Qué permite
DAC_READ_SEARCH	Bypass de lectura/búsqueda en directorios
LINUX_IMMUTABLE	Flags FS_IMMUTABLE / FS_APPEND en inodes
NET_BROADCAST	Broadcasts y escuchar multicasts
IPC_LOCK	Fijar páginas en RAM (mlock)
IPC_OWNER	Bypass de permisos en objetos IPC System V
LEASE	File leases (fcntl)
AUDIT_CONTROL	Configurar reglas de auditoría
MAC_OVERRIDE / MAC_ADMIN	Override de Mandatory Access Control (Smack)
WAKE_ALARM	Disparar alarmas que despiertan el sistema
BLOCK_SUSPEND	Bloquear la suspensión del host

```
docker run --cap-add=NET_ADMIN ... # sumar una
docker run --cap-drop=NET_RAW ... # quitar una del default
docker run --privileged ... # todas + acceso a devices
```

Kernel compartido: el trade-off

Container escape: explotar un bug del kernel → tomar el host

gVisor – kernel proxy en user space (Google)

Kata Containers – una micro-VM por container

Ambos son OCI-compatible → se intercambian en caliente en pipelines existentes.

Ninguno requiere cambiar el Dockerfile ni los scripts de CI: solo se cambia el runtime en la configuración del nodo.

Fuente: <https://gvisor.dev>

Quiz rápido

1. ¿Qué feature del kernel de Linux impone límites de memoria a los containers?
2. ¿Qué namespace hace que el proceso del container se vea a sí mismo como PID 1?
3. Nombrá una alternativa a runc que use un kernel en user space.

Resumen

- **Containers** = namespaces + cgroups + OverlayFS, no un kernel separado
- **Capabilities** = root partido en bits; Docker conserva 14 y dropea el resto
- **El runtime stack es intercambiable** porque la OCI estandariza el contrato
- **Kernel compartido** = la fuente de la eficiencia y también del trade-off de seguridad

Para seguir leyendo

- **OCI Runtime Spec** – opencontainers.org
- **gVisor** – gvisor.dev
- **Kata Containers** – katacontainers.io

¿Preguntas?